

Affiliate AE

Backend Purchase, Points & Redemption Flow

Client-facing technical overview of the current system behavior. The UI can change independently; the business rules and Supabase operations remain on the backend.

1. System overview

Affiliate AE connects customers and merchants through a customer QR code. A merchant scans the customer QR, the backend verifies the customer, and the merchant completes either a purchase transaction that issues points or a redemption transaction that applies a discount.

Actor	Responsibility
Customer	Shows the personal QR code and receives points, balance updates and notifications.
Merchant	Scans the QR, verifies the customer and chooses purchase/earn or redeem/discount.
Backend API	Validates identity, applies reward rules, writes transactions and queues notifications.
Supabase	Stores customers, balances, orders, redemptions, settings and notification logs.

2. Customer QR identification

- The customer app creates a QR payload containing the customer code (or customer UUID for compatibility).
- The merchant scanner accepts both the current JSON payload and legacy plain identifiers.
- The backend resolves the identifier against the customers table by customer_code or UUID.
- After verification, the response includes customer name, phone, code, balance and whether the customer is new to that merchant.
- If the identifier cannot be matched, the API returns "Customer not found"; this is an identity lookup issue, not a points calculation issue.

3. Purchase and earn-points flow

This is the normal purchase path used when a customer buys from a merchant.

Step	Backend action
1. Scan	Merchant scans the customer QR and submits the customer code.
2. Enter amount	Merchant enters a purchase amount. Current minimum: INR 100.

3. Select earn rate	Merchant selects the configured points per INR 100 option. Merchant caps/settings are applied.
4. Create purchase	POST /api/checkouts processes the purchase and writes the order and reward transaction with idempotency protection.
5. Update balances	Customer and merchant-specific reward balances are updated by the database transaction.
6. Notify	Push notifications are sent to customer and merchant. WhatsApp is queued/sent when configured.
7. Refresh UI	Dashboard, orders and balances are refreshed so new values appear.

4. Redeem-points-for-discount flow

This flow consumes points already available at the merchant and calculates a discount from the transaction amount.

- Merchant selects "Redeem points for discount".
- Merchant enters transaction amount and points to redeem. Current validation: amount minimum INR 100, points minimum 100 and maximum 1,000.
- Backend verifies the customer and checks the merchant-specific balance.
- Discount percentage = $(\text{points redeemed} / 100) \times \text{merchant discount rate per 100 points}$.
- Discount amount = $\text{transaction amount} \times \text{discount percentage} / 100$.
- A point_redemptions record is created, then global and merchant-specific balances are deducted.
- Customer receives the redemption result and an optional WhatsApp redemption message.

5. Reward settings and dynamic rules

- Admin settings define earn-rate options, redeem options, subscription settings and system defaults.
- Merchant settings can override earn points per ₹100 and redeem discount per 100 points.
- The UI reads these values from the backend, so settings changes do not require redesigning screens.
- Backend calculations are authoritative; the client UI only collects input and displays the result.

6. Notifications

- Purchase event: customer and merchant push notifications are triggered after a successful purchase.
- Redemption event: customer receives the redemption result; WhatsApp can use the configured redemption template.
- Registration event: welcome/QR and temporary-password WhatsApp templates are used when configured.
- WhatsApp failures are logged separately and should not undo a successfully committed purchase or redemption.

7. Current implementation note

The current backend exposes purchase/earn and redemption as two separate, validated operations. The UI can offer both choices after scanning. A single atomic transaction that simultaneously redeems points and issues new

purchase points would require an additional backend endpoint/database function; it should not be assumed from the current separate endpoints.

8. Client testing checklist

- Scan a valid customer QR and confirm customer name/code/balance appears.
- Complete an INR 100+ purchase with a selected earn rate; verify order, balance and notification.
- Redeem 100-1,000 points with sufficient balance; verify discount, deduction and history.
- Try invalid customer code, insufficient balance, amount below INR 100 and points outside limits.
- Repeat a request with the same idempotency key and confirm it does not create a duplicate purchase.
- Verify WhatsApp and push delivery independently; notification failure should remain visible in logs.

Summary

The backend logic is the stable foundation: scan and verify, validate rules, write the transaction, update balances, and notify. The UI can be redesigned to make this flow clearer without changing the underlying accounting logic.